

End-to-End Architecture Documentation

1. Project Overview

This project implements a complete end-to-end data pipeline for retail sales analytics, following the medallion architecture (Bronze → Silver → Gold) using Databricks with Delta Lake as the storage layer and Tableau Desktop as the BI tool.

1.1 Objectives

- Ingest raw retail transaction data into a Bronze Delta table without transformation
- Profile the source data to identify and document all data quality issues
- Cleanse, cast, and transform the data into a star schema in the Silver layer
- Build a Tableau dashboard that answers all seven business questions
- Demonstrate SCD Type 2 implementation for product listing price history

1.2 Technology Stack

Component	Technology
Data platform	Databricks (Unity Catalog, database: university)
Storage format	Delta Lake (ACID transactions, time travel, schema enforcement)
Processing	Apache Spark via PySpark — notebooks 01_bronze_ingest.ipynb, 02_silver_star_schema.ipynb
BI / dashboarding	Tableau Desktop — Extract mode via Databricks native connector
Data model	Navicat — star schema reference model (.nmodel)
Source data	sales_data_sampling.tsv — 2,000 rows, 20 columns, full year 2025

2. Business Questions

All pipeline and schema decisions are driven exclusively by the seven business questions defined in the project requirements. Every column in every table is justified by at least one business question — columns with no analytical purpose are explicitly excluded.

#	Business question	Key tables / columns
BQ 1	Total revenue and tax per Store, per Region on daily/ weekly/monthly basis	DIM_DATE, DIM_STORE (Store_Region), FACT (TotalSalesAmount, Tax_Amount)

#	Business question	Key tables / columns
BQ 2	Which product category and sub_category combinations are most frequently discounted?	DIM_PRODUCT (Product_Category, Product_SubCategory), FACT (DiscountAmount)
BQ 3	Which brand has the highest total sales volume in a specific store?	DIM_PRODUCT (Brand), DIM_STORE (Store_Name), FACT (TotalSalesAmount)
BQ 4	Validate: Total = (Unit_Price × Qty) – Discount + Tax	FACT (all measures)
BQ 5	Which brand had the highest total sales in North Region during Q3?	DIM_DATE (Quarter_Num), DIM_STORE (Store_Region), DIM_PRODUCT (Brand)
BQ 6	Track listing price (MSRP) changes per product	DIM_PRODUCT (MSRP, StartDate, EndDate, IsCurrent) — SCD Type 2
BQ 7	Track transaction price (actual price paid)	FACT (UnitPricePaid)

3. Source Data Profiling

Profiling was performed directly against the raw TSV file before any ingestion, using PySpark in the Bronze notebook. Findings are listed below:

3.1 Dataset at a Glance

Metric	Value	Metric	Value
Total rows	2,000 (1,999 data + 1 header)	Unique SKUs	71
Columns	20	Stores	12
Date range	Jan 1 – Dec 31, 2025	Customers	1,100
Regions	4 (North, Midwest, South, West)	Brands	8

3.2 Data Quality Issues Found

The following issues were identified during profiling.

Reason code	Rows	Decision
NULL_TRANSACTION_ID	2	No PK — cannot load to fact table. Quarantined.
INVALID_DATE	18	Two patterns: '2025-13-45' (impossible date) and 'June 12, 2025' (wrong format). Quarantined.
DUPLICATE_TRANSACTION_ID	22	10 Transaction IDs appear twice with different content per pair. Both rows quarantined.
ZERO_QUANTITY	5	Quantity_Sold = 0, Likely void/cancelled transactions. Quarantined.
NULL_CUSTOMER_NAME	9	Customer_ID intact. Name imputed to 'Unknown' in Silver.

3.3 Business Rule Validation

The formula:

“**Total_Sales_Amount = (Unit_Price × Quantity_Sold) – Discount_Amount + Tax_Amount**”

was validated against all 2,000 rows within a 2-cent floating point tolerance.

Result: 0 failures. The formula holds universally across the dataset.

3.4 MSRP Variation — SCD Type 2

Every SKU in the dataset has multiple distinct MSRP values. This confirms that listing price changes are frequent and systematic — SCD Type 2 on DIM_PRODUCT is implemented.

3.5 SKU Instability

The same Product_SKU maps to many different product names, brands, and categories. SKU-500 appears with 24 distinct product names across 3 categories and 6 brands. This means SKU cannot be used as the sole natural key for the product dimension. The SCD composite key was therefore defined as: (Product_SKU + Product_Name + Brand + Category + SubCategory).

4. Bronze Layer

Notebook: 01_bronze_ingest.ipynb

4.1 Design Principles

The Bronze layer is a faithful digital copy of the source file. The Bronze contract has three rules:

- All 20 business columns are read as StringType
- Column names standardized to snake_case, and three metadata columns added

4.2 Metadata Columns Added

Column	Purpose
_source_file	Tracks which file this row came from — useful when processing multiple monthly files in future
_load_ts	Timestamp of when this batch was ingested — enables re-run detection and auditing
_row_num	Original row position in the source file — helps reconstruct load order and debug issues

4.3 Post-Write Verification

Verification checks after every Bronze write:

- **Check 1:** Row count
- **Check 2:** Delta history — DESCRIBE HISTORY confirms the write operation was recorded

4.5 Target Table

```
university.bronze_sales_raw
```

5. Silver Layer

Notebook: [02_silver_star_schema.ipynb](#)

5.1 Design Principles

Three rules govern the Silver layer:

- Every data quality decision is documented and logged
- All columns are cast to correct analytical types

5.2 Type Casting

Column	From	To
date	STRING	DATE
msrp	STRING	DECIMAL(18,2)
unit_price	STRING	DECIMAL(18,2)
quantity_sold	STRING	INTEGER
discount_amount	STRING	DECIMAL(18,2)
tax_amount	STRING	DECIMAL(18,2)
total_sales_amount	STRING	DECIMAL(18,2)
customer_name	STRING	STRING

5.3 Data Quality Tagging

Every row is tagged with a `dq_status` (CLEAN or QUARANTINE) and a `dq_reason` code before any split occurs. Rejected rows are written to `silver_quarantine` with a `_quarantine_ts` timestamp.

5.4 Star Schema — Table Inventory

Table	Description
silver_dim_date	Distinct valid dates from clean transactions
silver_dim_product	All product versions including SCD history
silver_dim_store	All stores with surrogate key
silver_dim_customer	All customers with surrogate key
silver_fact_sales	Grain: one transaction line item
silver_quarantine	All rejected rows with reason codes and quarantine timestamp

6. Star Schema Design

6.1 Grain

The grain of the fact table is one transaction line item — each row in `silver_fact_sales` represents one retail transaction.

6.2 DIM_DATE

Column	Type
Date_Key	DATE
Day_Num	INT
Week_Num	INT
Month_Num	INT
Month_Name	STRING
Quarter_Num	INT
Year_Num	INT

6.3 DIM_PRODUCT — (SCD Type 2)

Column	Type
Product_SK	INT
Product_SKU	STRING
Product_Name	STRING
Product_Category	STRING
Product_SubCategory	STRING
Brand	STRING
MSRP	DECIMAL(18,2)
StartDate	DATE
EndDate	DATE
IsCurrent	BOOLEAN

SCD Type 2 composite key: (Product_SKU + Product_Name + Brand + Category + SubCategory). SKU alone maps to many unrelated products — using it alone would merge unrelated SCD histories.

6.4 DIM_STORE

Column	Type
Store_SK	INT
Store_ID	STRING
Store_Name	STRING
Store_Region	STRING

6.5 DIM_CUSTOMER

Column	Type
Customer_SK	INT
Customer_ID	STRING

6.6 SALES_FACT

Column	Type
Transaction_ID	STRING
Date_Key	DATE
Product_SK	INT
Store_SK	INT
Customer_SK	INT
QuantitySold	INT
UnitPricePaid	DECIMAL(18,2)
DiscountAmount	DECIMAL(18,2)
Tax_Amount	DECIMAL(18,2)
TotalSalesAmount	DECIMAL(18,2)

UnitPricePaid is the only non-additive measure in the fact table.

7. Silver Layer Verification

Post-write verification checks are run at the end of 02_silver_star_schema.ipynb:

- **Check 1 — Row conservation:** fact_count + quarantine_count must equal bronze_count
- **Check 2 — FK integrity:** All four FK columns in the fact table (Date_Key, Product_SK, Store_SK, Customer_SK) must have zero null values.
- **Check 3 — Formula validation:** Total = (UnitPricePaid × QuantitySold) – DiscountAmount + Tax_Amount must hold for all fact rows within 2-cent tolerance.
- **Check 4 — SCD integrity:** Each (Product_SKU + Product_Name + Brand) combination must have exactly one row where IsCurrent = True.

8. Pipeline Architecture Summary

Layer	Input	Output
Bronze	sales_data_sampling.tsv	university.bronze_sales_raw
Silver	university.bronze_sales_raw	5 Delta tables + quarantine
BI	Silver tables via Databricks connector	SalesAnalysis_DAMG7370.twbx